ARI2204: Reinforcement Learning
Course Project
2025

Report

Daniela Curmi, 0058805L
Matthew Privitera, 0393704L
Matthew Pirotta, 0297505L

Table of Contents

Introduction 3
Blackjack Environment & RL Framework 4
RL Implementations and Analysis 5
- Total Unique State-Action Counts across all algorithms 5
- Monte Carlo On-Policy Control (First visit) 6
- SARSA On-Policy Control 10
- Q-Learning (SARSAMAX) Off-Policy Control 14
- Double Q-Learning Off-Policy Control 17
- Combined Win Rate 21
- Dealer Advantage 24

Conclusion 25
Generative AI Usage 26
- Introduction 26
- AI Usage 26
- Critical Assessment 29
- Conclusion 29

References 30
Plagiarism Form 31
Distribution of Work 32

Introduction

Blackjack is a widely played casino card game in which players attempt to accumulate a hand value as close to 21 as possible without exceeding it. In this game, face cards are valued at 10, while aces can flexibly count as either 1 or 11. In the standard version studied in reinforcement learning (RL) literature, players compete independently against a dealer [1]. Play begins with two cards dealt to both player and dealer, with one of the dealer's cards being visible. If the player obtains a hand totaling exactly 21 with the first two cards (natural), they win immediately unless the dealer also holds a natural in which case it results in a draw. If the player does not have a natural, they may draw additional cards (hit) until either they stop (stand) or exceed 21 (bust). Dealers follow a fixed policy where they must hit until reaching a hand value of 17 or more after which they must stand. If neither party busts, the winner is the one whose hand is closer to 21 without exceeding it. This probabilistic, sequential decision-making structure presents an ideal application for reinforcement learning techniques.

This project aims to develop a simple Blackjack environment and implement four distinct RL algorithms to train agents within this environment: First-Visit Monte Carlo Control, SARSA, Q-Learning and Double Q-Learning. Each agent is trained and evaluated under various ε-greedy exploration strategies, both with and without exploring starts when applicable. Agent performance is evaluated across 100,000 episodes where metrics such as win/ loss/ draw rates, strategy convergence, state-action pair coverage and dealer advantage are measured.

The report is structured into three main sections. The first section details the design specifications of the Blackjack environment, state abstraction and the architecture of the reinforcement learning framework. The second section examines the implementation of each learning algorithm by providing an analysis of their implementation and performance characteristics. The final section observes the performance of each algorithm by evaluating the dealer's statistical advantage and provides concluding insights on the effectiveness of these reinforcement learning approaches in playing Blackjack optimally.

Blackjack Environment & RL Framework

The *Deck* class handles card generation, shuffling, and drawing operations with a simplified representation where face cards (J, Q, K) are directly represented as '10'. The *Hand* class maintains cards for players and dealers while implementing game logic like calculating hand totals, detecting busts, and managing the dual value of aces. The core *Blackjack* class controls gameplay by managing state transitions, executing player actions (hit/stand), implementing the dealer's fixed policy (hit until 17+), and determining game outcomes with appropriate rewards (+1 for win, -1 for loss, 0 for draw). The environment represents the game state as a tuple of player total, dealer's visible card, and a flag indicating whether the player has a usable ace. This implementation sets the foundation for an easy to use interface when implementing the reinforcement learning framework.

For efficient storage of Q-values and counts, a DefaultDict was used. Its O(1) time complexity for lookups, insertions, and updates enables quick access to state-action values during policy evaluation and improvement. The automatic initialization of new keys also simplifies the code by removing the need for conditional checks.

The known game state consists of player total, dealer's visible card, and usable ace flag. The player total tells the algorithm how close it is to 21 and busting. The dealer's visible card gives an indication of how risky the player has to play, whether to risk hitting on higher player totals or playing it safe and standing. Additionally, the dealer's visible card, if an ace, is considered as a static 11 instead of a 1 or 11, forcing the algorithm to play more aggressively since the dealer is in an advantageous position. Lastly, the usable ace flag allows the player to make more risky plays as there is a low chance of busting when recounting the ace as a 1 instead of an 11 [2].

A single episode is defined as a single game of Blackjack from start to the terminal state. As specified in the project brief, a discount factor was not used since the episodes are very short. All RL algorithms used in the project make use of a custom ε-Greedy policy. The player has a static policy for trivial solutions: always hitting under 12 since it is guaranteed not to bust, and standing on 21 since no better state can be achieved. For the remaining range of [13,..20], the typical ε-Greedy policy is employed, where the policy chooses the action with the best Q-value for a given state [2]. However, there is also a chance based on an epsilon value that the policy will choose a random action instead, exploring the possible action space. The epsilon value calculation is set as a parameter, experimenting with different tradeoffs of exploration and exploitation over the course of multiple episodes.

RL Implementations and Analysis

Total Unique State-Action Counts across all algorithms

Before evaluating the learned strategies and their performance, it is important to first verify whether the agents sufficiently explored the environment. The state space has 9 (player_total) × 10 (dealer_card) × 2 (usable_ace) = 180 states. The agent then has 2 possible actions, hit or stand, for a total of 360 state-action pairs.

Across all agent models and configurations, each of these 360 state-action pairs was visited at least once during training. An example plot for the SARSA algorithm is presented in Fig.1, where the bar chart displays the frequency of visits under four ε-greedy configurations, i.e. ε = 0.1 (fixed), ε = 1/k (inv), ε = $e^{-k/1000}$ (exp_fast), and ε = $e^{-k/10000}$ (exp_slow). The uniform convergence suggests that all the algorithms, with each ε-decay setting, eventually explored the entire relevant space, which is important since the nature of the algorithm depends on updating Q-values based on the actions it takes. Overall, none of the configurations were too greedy too early, meaning that algorithms maintained on/off-policy coverage without prematurely converging to suboptimal exploitation.

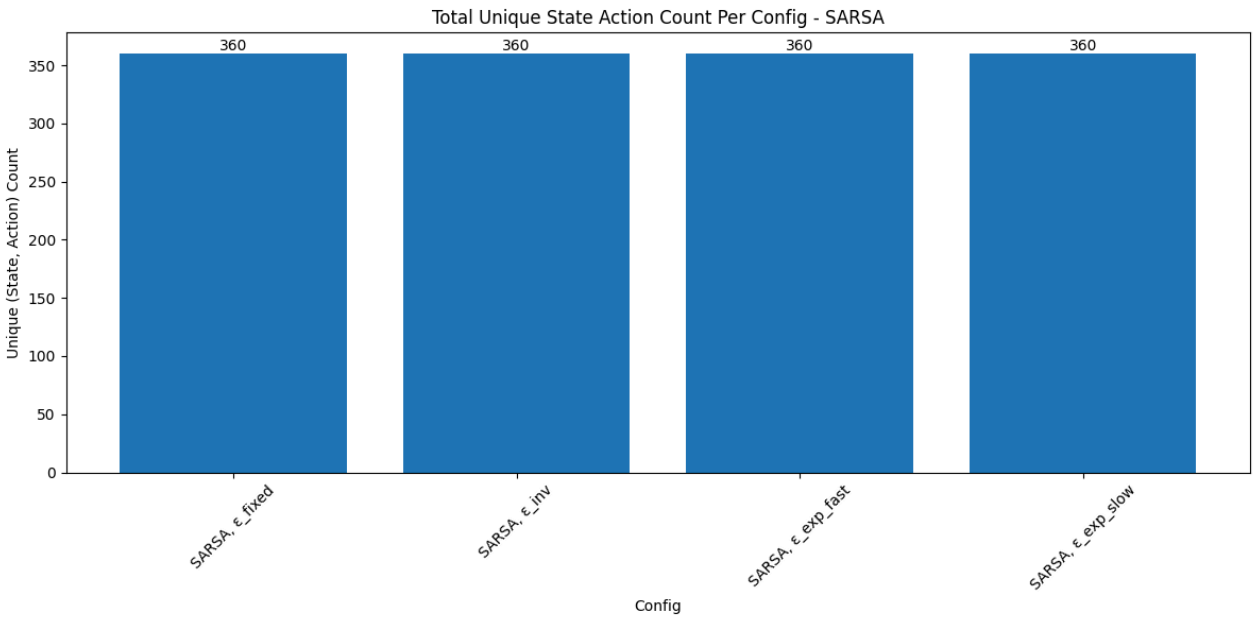

Fig.1 Bar chart that counts each unique state-action pair per configuration

Monte Carlo On-Policy Control (First visit)

➔ *Algorithm Overview and Implementation*

Monte Carlo (MC) methods are model-free reinforcement learning algorithms that learn from complete episodes, making them ideal for games like Blackjack where the MDP is unknown. Unlike dynamic programming, MC doesn't need a transition model and instead learns directly from experience.

In this project, MC was implemented by first generating an entire episode and then performing a backward pass to propagate the terminal reward to all state-action pairs visited along the way. There are two main variants of MC: first-visit and every-visit. First-visit MC updates the value of a state-action pair only the first time it is encountered in an episode, providing an unbiased estimate. Conversely, every-visit MC updates on every occurrence of a state-action pair, which can reduce variance but introduce some bias. For Blackjack, the first-visit approach was chosen, although in this particular case the choice is arbitrary since Blackjack episodes do not contain cycles, once a state is visited, it is not revisited within the same game [3].

A possible Monte Carlo configuration is 'Exploring start' which ensures that all state-action pairs are visited by randomly selecting the first non trivial action in each episode. This promotes sufficient exploration, a key requirement for Monte Carlo control to converge to the optimal policy. In Blackjack, it is especially effective due to the episodic nature of the game, allowing the agent to learn from a diverse set of starting conditions.

➔ *Graph Analysis and Results*

Different exploration and exploitation strategies significantly affect the frequency with which state-action pairs are visited during training. As shown in Fig. 2, the configuration ***'MC, Es, ε_inv'***, which incorporates exploring starts by design, results in a relatively uniform distribution of state-action visit counts. The ***'MC, NoEs, e_exp_slow'*** configuration displays a similar explorative pattern, despite not using exploring start. This outcome aligns with expectations, as the slower exponential decay of ε encourages sustained exploration over a longer period, increasing the likelihood of visiting a wider variety of state-action pairs [3].

In contrast, the **'MC, NoEs, ε_inv'** configuration shows a markedly different distribution compared to **'MC, Es, ε_inv'**, despite the only difference being the absence of exploring starts. Without the initial forced exploration, **'MC, NoEs, ε_inv'** exhibits a skewed distribution, favoring the selection of the current best-known actions, which suggests a faster shift toward exploitation. Similarly, **'MC, NoEs, ε_exp_fast'** also displays a skewed pattern due to its rapidly decaying ε, reinforcing exploitation. However, its distribution is slightly less concentrated than that of **ε_inv**, indicating a better balance between exploration and exploitation in early episodes.

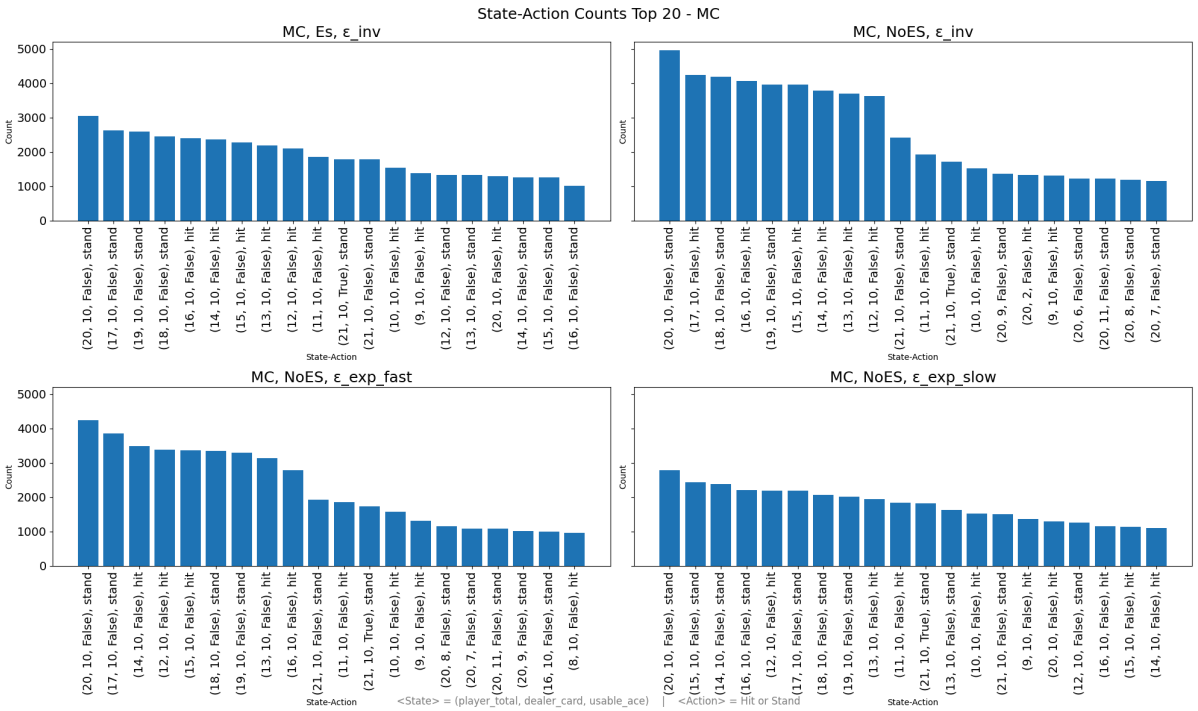

Fig.2 Bar charts of the top 20 state-action pairs for each Monte Carlo configuration

With a better understanding of how the different configurations handle the exploration vs exploitation tradeoff, the resulting strategy tables offer valuable insight. Across all configurations, the learned policies generally follow expected behavior, standing on higher player sums and hitting on lower ones [4]. Furthermore, when operating with a usable ace state, the algorithms tend to adopt a more aggressive playstyle, extending the range of sums for which they

choose to hit. This behavior reflects an effective use of the flexibility offered by the Ace. However, several **non-usable Ace** strategy tables reveal some counterintuitive decisions. In particular, certain configurations recommend standing with lower player totals (12-13) against weak dealer upcards (4-6), where basic Blackjack strategy would typically suggest hitting [5].

The exploring starts variant, '***MC, Es, ε_inv***' demonstrates clearer decision boundaries compared to its non-exploring counterpart, showing more structured patterns regarding hit versus stand decisions. As evidenced in Fig. 3, the '***MC, NoEs, ε_inv***' configuration had the largest most exploited cell count. This overexploitation and under exploration likely accounts for the inconsistent "holes" observed within its strategy table, where isolated decisions contradict the surrounding pattern.

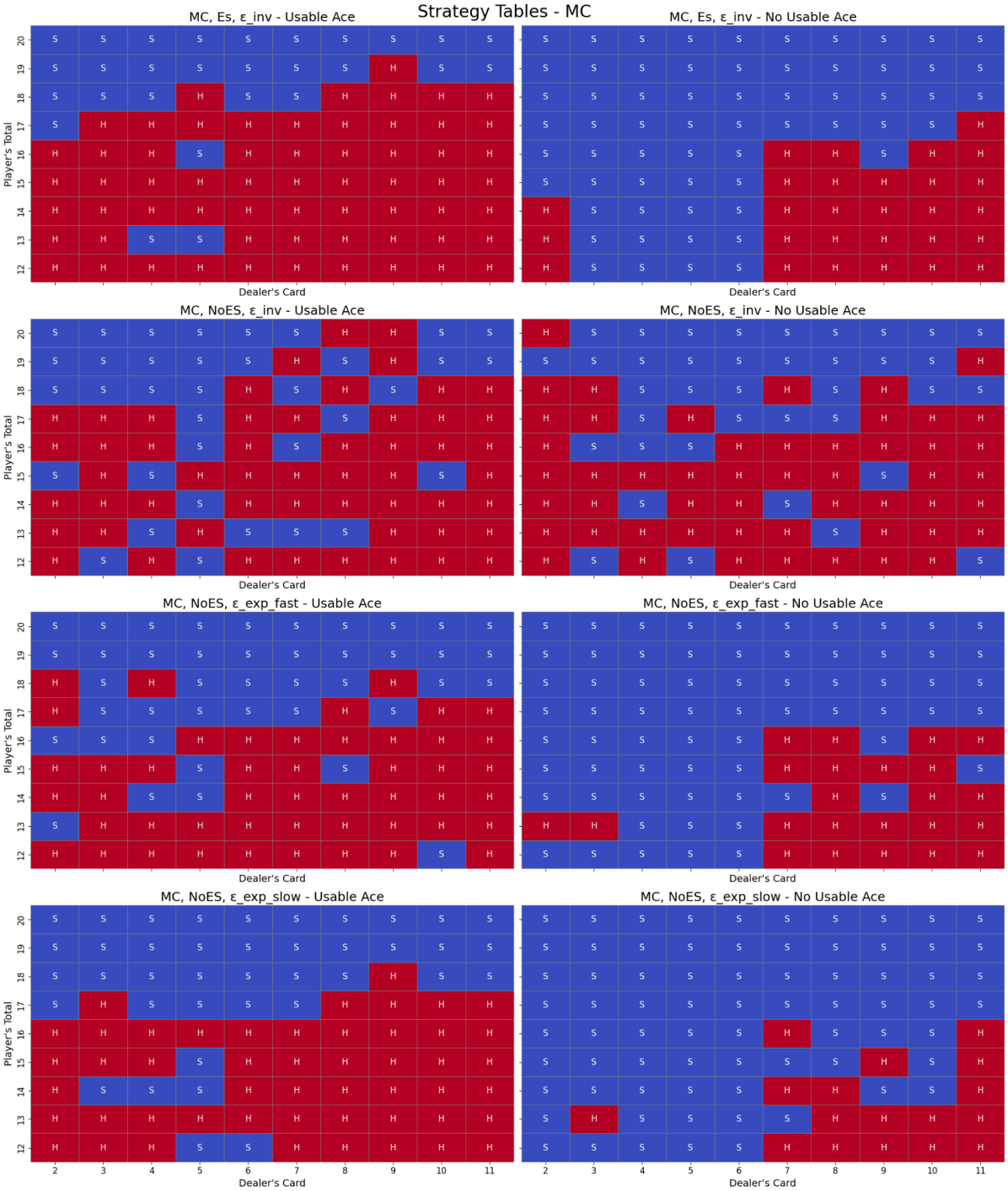

Fig.3 Strategy table for each Monte Carlo configuration

SARSA On-Policy Control

➔ *Algorithm Overview and Implementation*

The SARSA algorithm is an on-policy temporal-difference (TD) control algorithm used in RL to learn the optimal action-value function **Q(s,a)** for a given environment. Unlike value prediction methods that estimate the value of states, SARSA directly estimates the value of state-action pairs while following the same policy it is evaluating, hence the term on-policy. This makes it especially compatible with exploration policies, such as the epsilon-greedy policy used throughout this assignment. A single step in an episode consists of the tuple **(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})**, where we start in the state **S_t**, take the action **A_t**, and receive the reward **R_{t+1}**. Then we transition to the state **S_{t+1}** and take the next action **A_{t+1}** based on the same policy [1].
The update rule for the SARSA implementation is:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \Big[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) \Big]$$

where **α** is the step size and **γ** is the discount factor (in this case, no discount factor is assumed i.e. **γ=1**). This rule updates the current estimate **Q(S_t, A_t)** using the observed reward and the estimated future value **Q(S_{t+1}, A_{t+1})**. Hence, the next action **A_{t+1}** is chosen from the current behavior policy, not necessarily the greedy one. In contrast, off-policy methods like Q-learning always use the maximum estimated value of the next state, regardless of the policy used to choose actions. In the context of an ε-greedy policy, SARSA balances exploration and exploitation by occasionally choosing a random action with probability **ε**, and otherwise choosing the action with the highest estimated value. As learning progresses, ε decays over time via four different configurations i.e., **ε=0.1, ε=1\k, ε=$e^{-k/1000}$, ε=$e^{-k/10000}$**, encouraging more exploitation of learned knowledge while still allowing sufficient exploration. In order to implement the SARSA algorithm, we used the following pseudo-code as the core structure of the algorithm in Fig. 4, as explained above [5], [10].

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S
  Choose A from S using policy derived from Q (e.g., ε-greedy)
  Loop for each step of episode:
    Take action A, observe R, S'
    Choose A' from S' using policy derived from Q (e.g., ε-greedy)
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma Q(S', A') - Q(S, A)\big]$
    $S \leftarrow S'$; $A \leftarrow A'$;
  until S is terminal

Fig.4 Sutton and Barto (2018) Chap. 6 p.130

Therefore, the convergence of SARSA to the optimal policy is guaranteed if the step size **α** decays appropriately, all state-action pairs are visited infinitely often, and if the policy becomes greedy in the limit. Overall, SARSA is ideal for environments where it's important to evaluate the actual performance of the current policy, including its exploratory actions, rather than always assuming optimal behavior [5].

➔ *Graph Analysis and Results*

State-Action Counts

To evaluate the depth of learning, the plots in Fig.5 were used to observe skew, revealing distinct patterns in how exploration and exploitation were balanced. Some state-action pairs dominate across all configurations, particularly those with mid-to-high player totals and visible dealer cards, indicating that these scenarios occur frequently or are favored by the learned policy. The ε_fixed and ε_inv configurations show more skewed distributions, with the highest-ranked pairs appearing significantly more often, suggesting faster convergence toward exploitation. On the other hand, the exponential decay configurations display a flatter distribution, indicating more sustained exploration throughout training. These results show that while all configurations achieve full state-action coverage, the structure of the frequency distribution is shaped by both the decay schedule and the underlying dynamics of the game itself. In general, sustained exploration leads to broader state-action visitation, while bolder decay schedules concentrate updates on high-value regions of the policy space.

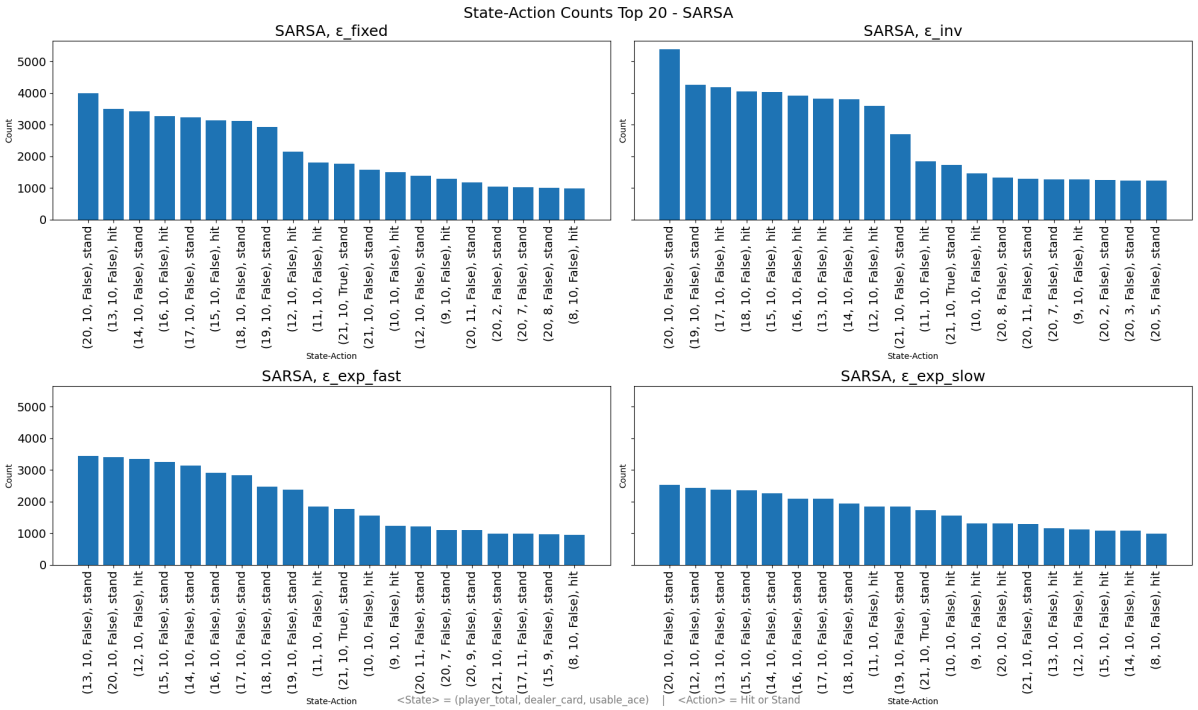

Fig.5 Bar charts of the top 20 state-action pairs for each SARSA configuration

Strategy Tables

To analyse the overall performance, the strategy tables in Fig.6 across most of the ε-decay configurations reveal consistent and reasonable decision boundaries that align closely with the usual Blackjack strategies. However, without a usable ace across all configurations, the agent opts for more conservative play, choosing to "stand" more often as a result, especially when the player's total is high, 19–21. In contrast, with a usable ace, the agent adopts a more aggressive approach, hitting more frequently even at intermediate totals like 15–17. This is expected, as a usable ace provides flexibility and reduces the risk of busting. Among the configurations, ε_fixed and ε_exp_fast produce the cleanest and most stable policies with clearly defined "hit" and "stand" regions, indicating strong convergence. Meanwhile, ε_inv and ε_exp_slow exhibit scattered behavior, especially when there's no-usable-ace, suggesting less consistent learning due to continued exploration or slower exploitation. In general, SARSA reliably learns reasonable policies in both cases, but the exploration schedule significantly impacts the clarity and optimality of the learned strategies.

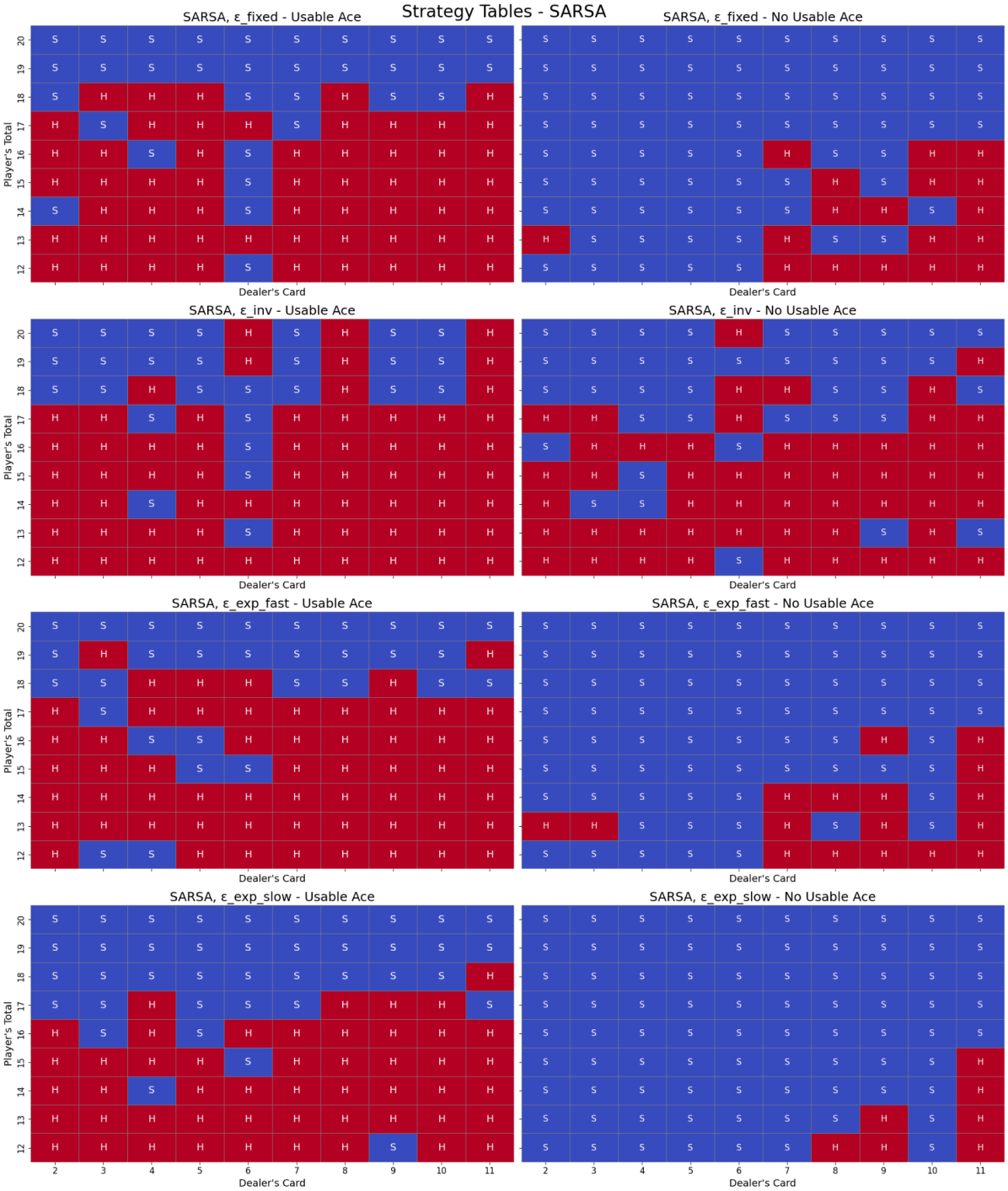

Fig.6 Strategy tables for usable and non-usable ace scenarios per SARSA configuration

Q-Learning (SARSAMAX) Off-Policy Control

➔ *Algorithm Overview and Implementation*

The Q-learning (SARSAMAX) algorithm is an off-policy TD control algorithm that estimates the action-value function **Q(s,a)** like SARSA, but differs in how it handles action selection and policy evaluation. While SARSA updates Q-values based on the action taken under the current policy, Q-learning assumes the agent will act optimally by always selecting the action with the highest estimated value, regardless of the behavior policy used to gain experience. This makes Q-learning bolder in terms of exploitation of learned knowledge since the update rule uses the maximum Q-value at the next state, which effectively separates learning from exploration [5].
The update rule for the Q-Learning implementation is:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\Big]$$

Although an ε-greedy policy is still used to create exploratory behavior, Q-learning learns as if the agent were always following a greedy policy. This approach often leads to faster convergence to the optimal policy but may be less robust in stochastic environments where exploration behavior affects long-term performance. To implement the Q-Learning algorithm, we followed the structure of the pseudo-code in Fig.7 [6],[8],[10].

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose A from S using policy derived from Q (e.g., ε-greedy)
    Take action A, observe R, S'
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma \max_a Q(S', a) - Q(S, A)\big]$
    $S \leftarrow S'$
  until S is terminal

Fig.7 Sutton and Barto (2018) Chap. 6 p.131

➔ *Graph Analysis and Results*

State-Action Counts

The top 20 state-action count plots in Fig.8 reveal how different ε-decay schedules impact the agent's balance between exploration and exploitation. Across all configurations, state-action pairs involving high player totals, 20 or 21, and strong dealer cards, 10, dominate, reflecting frequent game dynamics and the agent's preference for exploiting high-value decisions. ε_inv configurations

exhibit the steepest drop-offs in visitation counts, with the top state-action pairs occurring more frequently than others. This skew suggests earlier convergence to policies that prioritise exploitation. In contrast, the slow exponential configurations show flatter distributions, indicating persistent exploration throughout training, enabling the agent to revisit a wider range of pairs more evenly. Overall, these patterns confirm that more aggressive decay leads to efficient policy refinement in dominant states, whereas slower decay facilitates broader coverage and potentially more robust policies in stochastic scenarios [7].

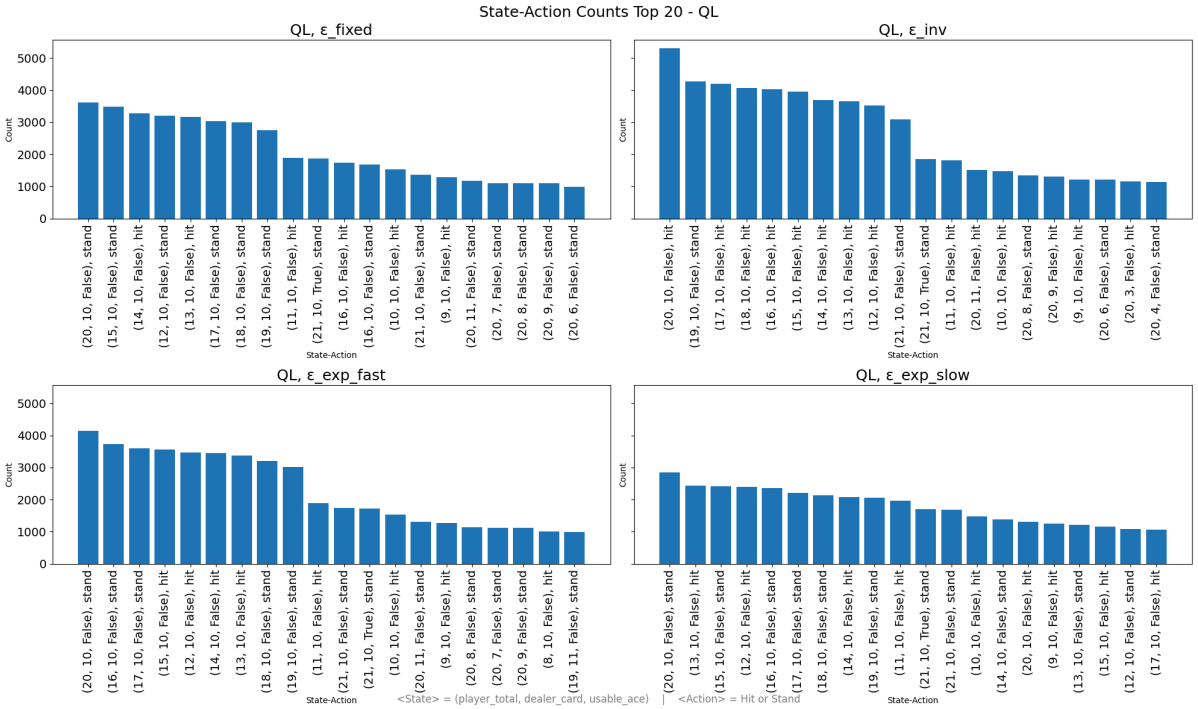

Fig.8 Bar charts of the top 20 state-action pairs for each Q-learning configuration

Strategy Tables

The Q-learning strategy tables across all ε configurations in Fig.9 reveal that while the algorithm generally learns sensible policies, its stability is more sensitive to the ε-decay schedule compared to SARSA. Across all configurations, the distinction between usable and non-usable ace cases remains consistent, as the policy exhibits behavior similar to that of SARSA. Similarly, ε_fixed and ε_exp_fast show the most coherent strategy boundaries which mirror optimal Blackjack behavior and suggest effective convergence under moderate exploration. In contrast, QL_ε_inv and QL_ε_exp_slow produce noisier policies, especially with no usable ace, where the action choices vary irregularly across neighboring states. This inconsistency stems from prolonged exploration interfering with stable policy refinement. Overall, Q-learning demonstrates the capacity to learn high-quality policies in Blackjack, but its performance is strongly tied to how quickly it reduces exploration [7].

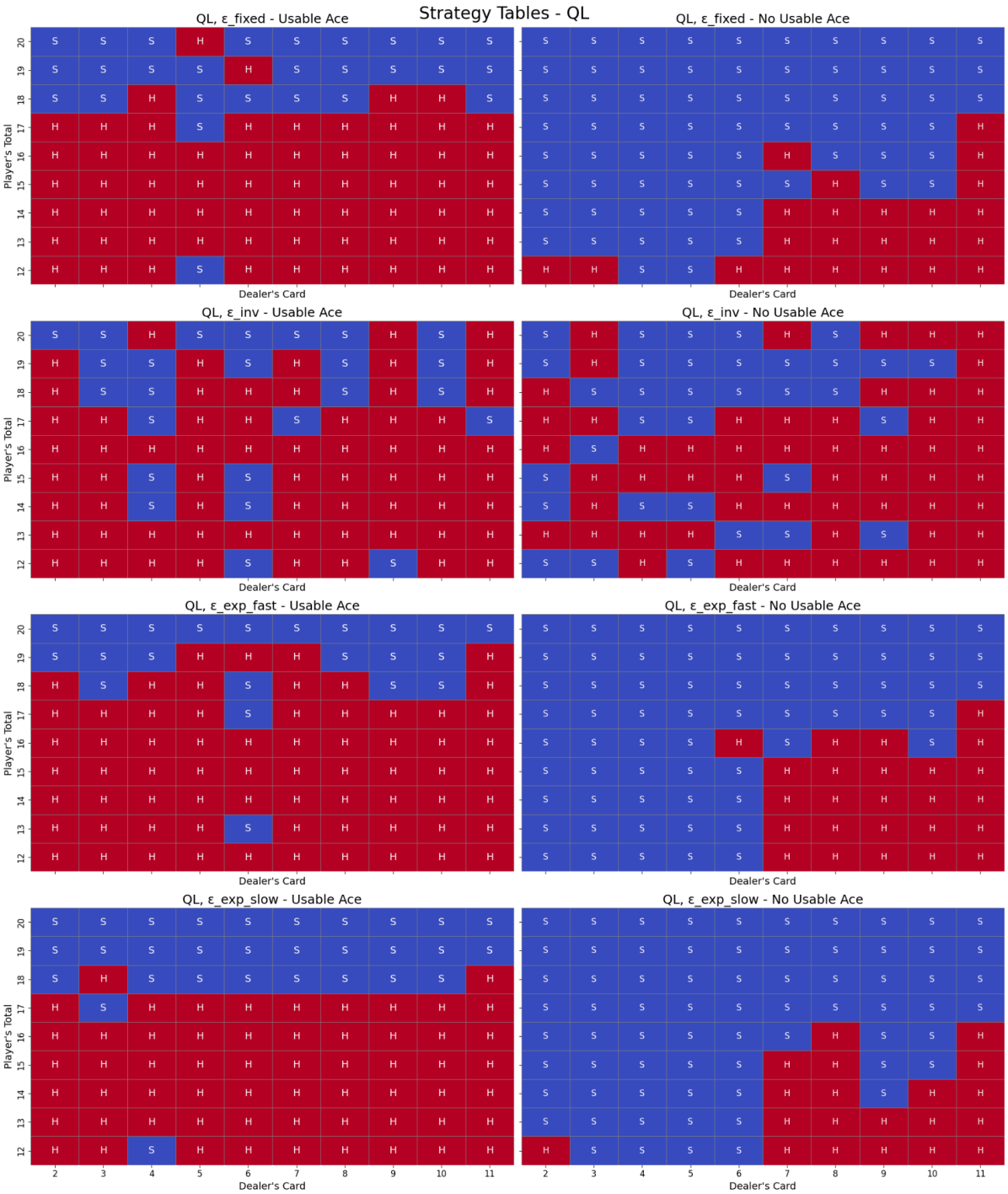

Fig.9 Strategy tables for usable and non-usable ace scenarios per Q-Learning configuration

Double Q-Learning Off-Policy Control

➔ *Algorithm Overview and Implementation*

Double Q-learning is an off-policy TD control algorithm and is an extension of Q-learning to address overestimations of the action values [9]. In standard Q-learning, the TD target is calculated using the maximum action-value from the current Q-table which can result in upwardly biased value estimates when action values are unbalanced. Double Q-learning addresses this by decoupling the action selection from the action evaluation by using two separate Q-value estimators, Q^A and Q^B. At each step, a single Q-table has a 50% chance to be updated. The action that maximises the value from the selected Q-table is used for action selection whereas evaluation is performed using the other Q-table. This introduces stochasticity in the updates and prevents consistent overestimation of action values [1].

The update rules for Double Q-Learning are as follows. If Q^A is selected to be updated, then:

$$Q^A(s,a) \leftarrow Q^A(s,a) + \alpha(s,a)\left(r + \gamma Q^B(s',a^*) - Q^A(s,a)\right)$$

Otherwise, if Q^B is selected:

$$Q^B(s,a) \leftarrow Q^B(s,a) + \alpha(s,a)(r + \gamma Q^A(s',b^*) - Q^B(s,a))$$

In the project, Double Q-learning is implemented within the **RLAlgorithms.double_q_learning()** method which manages the learning loop for each episode. Two separate Q-tables (_Q and _Q2) are maintained using DefaultDict structures and action selection is performed using an ε-greedy strategy based on the sum of both Q-values. At each step, one of the two Q-tables is randomly chosen for updating while the other is used to evaluate the value of the best action chosen from the first table. This separates action selection from value estimation and reduces the problem of overestimation bias. Updates are applied using an incremental learning rate, $\boldsymbol{\alpha} = \frac{1}{N(s,a)}$, with visit counts tracked in _N. To avoid learning from trivial states, updates are conditionally restricted to player totals between the interval [12,20]. After training, the final policy is derived by averaging the corresponding values from both Q-tables for a stable estimate of the agent's learned strategy. [9]

➔ *Graph Analysis and Results*

State-Action Counts

Across all ε-decay configurations in Fig.10, the most frequently visited state-action pair is consistently (20, 10, False), stand, indicating that the agent learned to consistently stand when holding a total of 20 against a dealer 10. The high frequency of this state-action shows convergence to an optimal or near-optimal policy in that region of the state space which aligns with real expected strategies in Blackjack.

Under 'DQL, ε_fixed', the agent's behaviour exhibits reasonable exploration-exploitation balance. The more evenly distributed visitation pattern suggests that the agent maintains consistent exploration throughout training by potentially sacrificing some convergence speed for a larger state space coverage. This balanced approach may produce a more robust policy across the full state space rather than overexploiting in a few dominant states. With 'DQL, ε_inv', the agent has committed to a small subset of dominant strategies evidenced by the disparity between the highest-frequency and lowest-frequency pairs. This shows that exploration rapidly diminishes in favour of exploiting discovered strategies. 'DQL, ε_exp_fast' follows the same pattern as that of 'DQL, ε_inv' where the agent has also identified state-action pairs to exploit while still only exploring fewer common states. Lastly, 'DQL, ε_exp_slow' shows the flattest overall distribution with lower counts across all state-actions. This is the most balanced exploration-exploitation trade-off among all variants, potentially creating a more generalised policy. The gradual decline shows that the agent maintains meaningful exploration throughout the episodes while still converging toward optimal strategies.

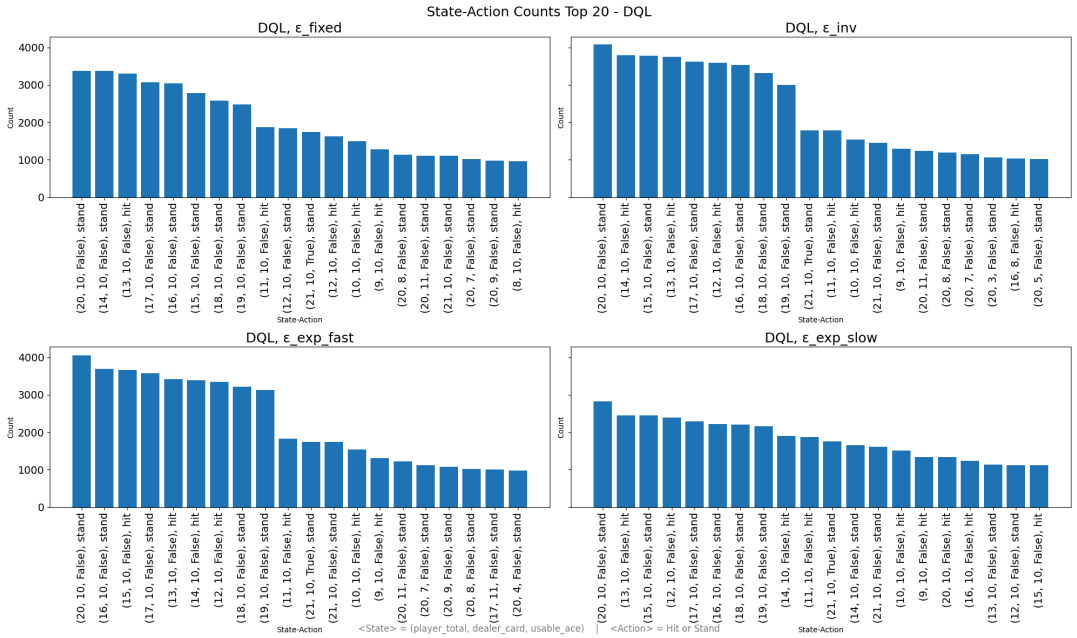

Fig.10 Bar charts of the top 20 state-action pairs for each Q-Learning configuration

Strategy Table

The strategy tables seen in Fig.11 reveal consistent patterns across different ε-decay configurations. Firstly, there is a clear distinction between strategies for hands with and without usable aces. In scenarios with usable aces, the policies yet again show more aggressive hitting behaviour especially in the middle range of player totals (12-16) which aligns with traditional Blackjack strategy. Across all configurations, strong convergence toward standing on high totals (19-21) can be observed regardless of the dealer's card, demonstrating the algorithm's ability to learn fundamental Blackjack principles. However, the ε_exp_fast configuration yields nosier strategies especially in the 'usable ace' region, probably due to overcommitment to early learned policies.

The policies consistently show more conservative play when facing lower dealer cards (2-6) compared to higher cards (7-A), so the agent is aware of the risks in the no-usable ace scenario. As such, standing becomes more prevalent when player totals reach 17 or higher where the agent always chooses to stand. However, all configurations seem to struggle with similar edge cases when player totals are around 16-17 against the dealer's 7-8 where the optimal strategy is non-trivial. Moreover, where typical Blackjack strategy says to hit on low hands around 12–13 against lower dealer cards of 2-5, the agent still decides to stand. This behaviour was also observed in other algorithms. These occasional inconsistencies in these regions suggest that even with different exploration approaches, certain state-action values remain challenging to differentiate due to their close expected returns in the Blackjack game.

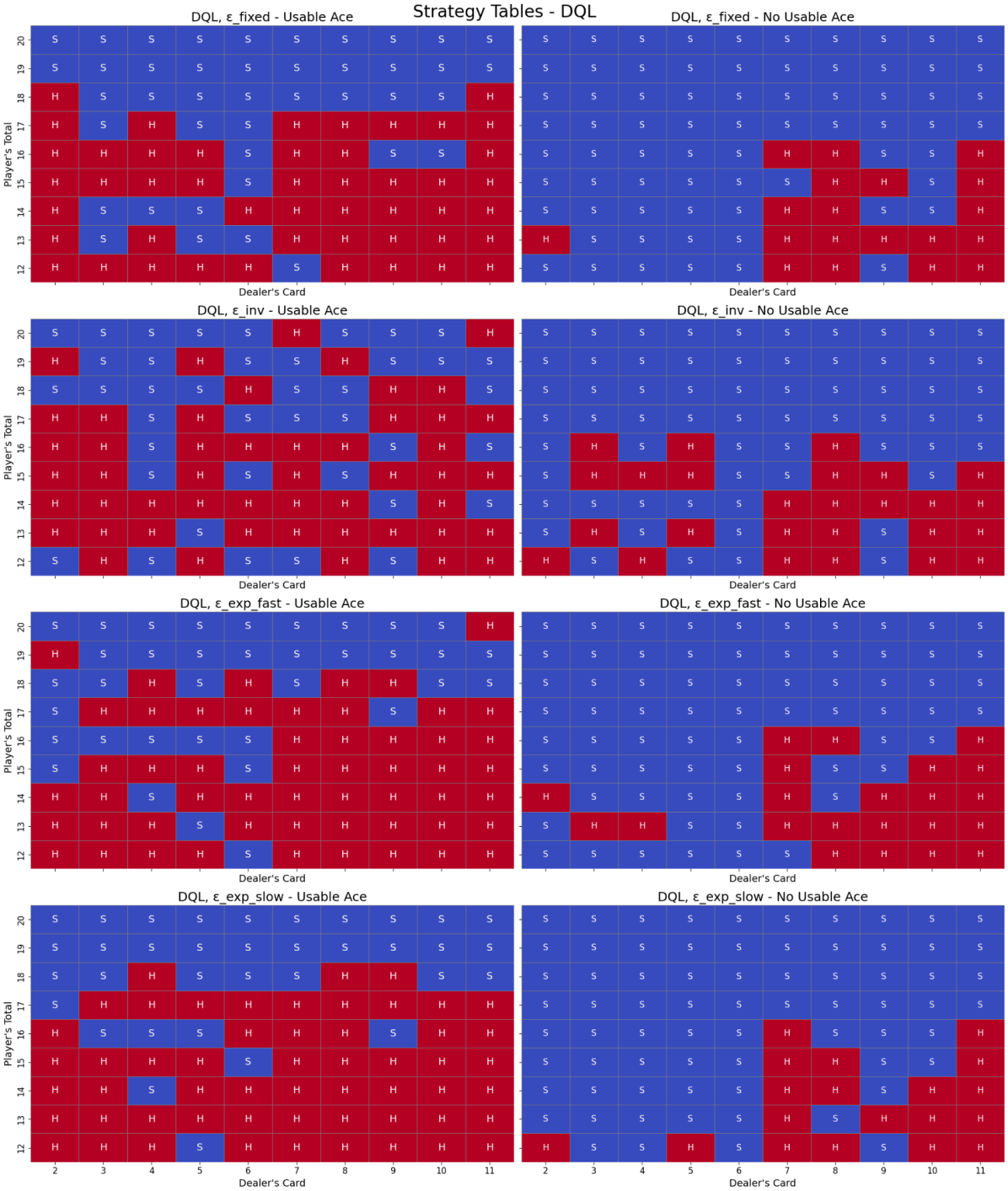

Fig.11 Strategy tables for usable and non-usable ace scenarios per Double Q-Learning configuration

Combined Win Rate

The overall performance trends for Monte Carlo (MC), SARSA, and Q-Learning algorithms reveal consistent patterns in how different exploration strategies influence learning in the blackjack environment (see Figs. 12, 13, 14, 15). Across all three algorithms, the ε_inv configuration generally underperforms, yielding lower win rates (typically 320–390) and higher losses (520–620), suggesting that its aggressive decay prematurely limits exploration. Conversely, ε_fixed maintains stable outcomes with moderate win rates (400–430) and loss rates near 500, indicative of early policy convergence but limited late-stage improvement.

The ε_exp_fast strategy consistently shows the most dynamic learning behaviour across algorithms, with clear increases in win rates, from 320 up to 430 and corresponding decreases in losses. This demonstrates an effective transition from early exploration to later exploitation. In contrast, ε_exp_slow, while starting with the weakest performance (as low as 300 wins and 600+ losses), shows steady and consistent improvement over time. This highlights its potential for long-term learning benefits despite slow initial gains.

Monte Carlo results reflect similar trends. The Es (Exploring Starts) variant performs slightly better than NoES (No Exploring Starts), though both configurations show modest improvements over time. Once again, ε_exp_fast exhibits the most noticeable learning trajectory, reinforcing the advantage of a well-balanced decay rate in episodic settings like Blackjack.

Draw rates remain largely unaffected by algorithm type or exploration strategy, staying within a narrow and stable range (60–100 counts). This suggests that while exploration strategies influence win/loss outcomes, they have minimal impact on the likelihood of games ending in draws.

Overall, ε_exp_fast emerges as the most effective strategy across all methods, offering a strong compromise between early exploration and late exploitation. ε_inv proves too aggressive, stalling learning prematurely, while ε_exp_slow trades early performance for gradual long-term gains. These patterns underscore the importance of tuning exploration decay to align with the learning dynamics of the environment and algorithm used.

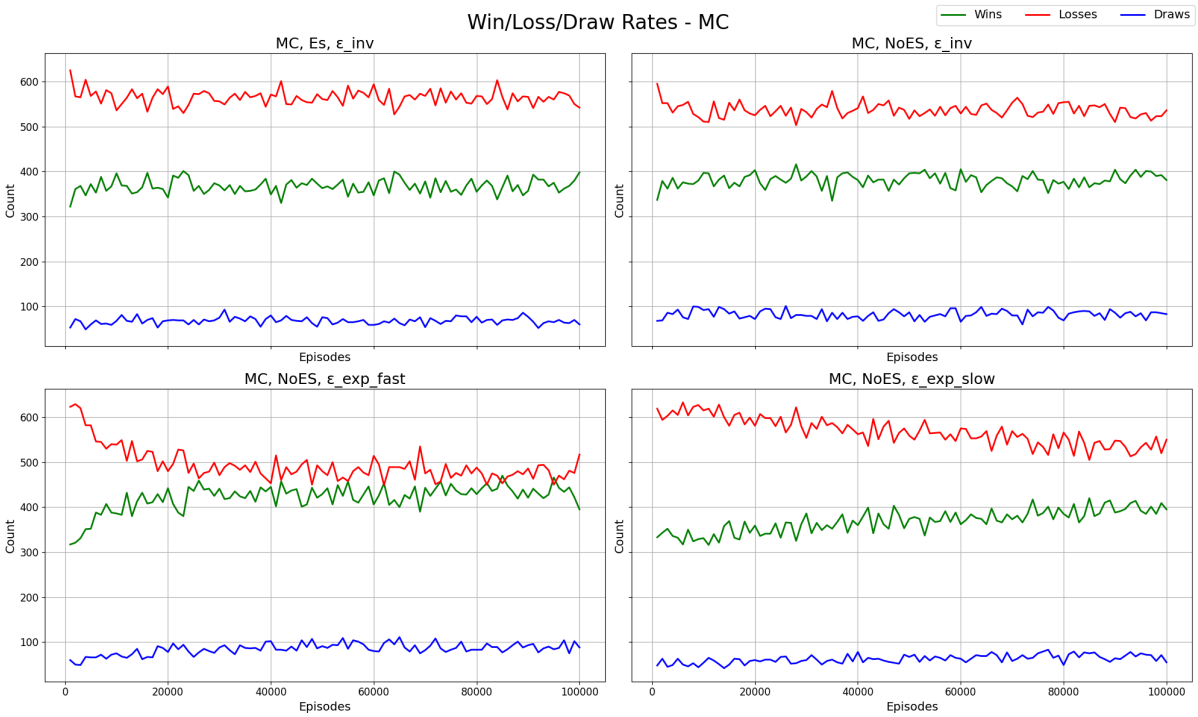

Fig.12 Win/Draw/Loss Rates for MC for all configurations

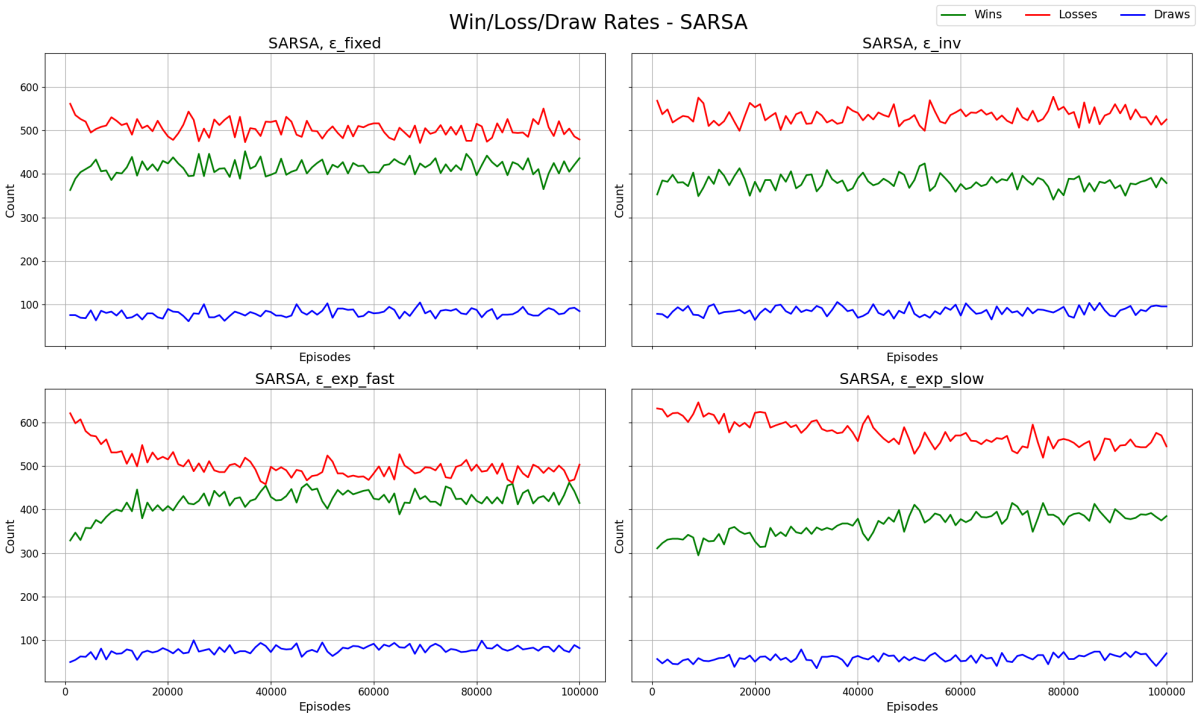

Fig.13 Win/Draw/Loss Rates for SARSA for all configurations

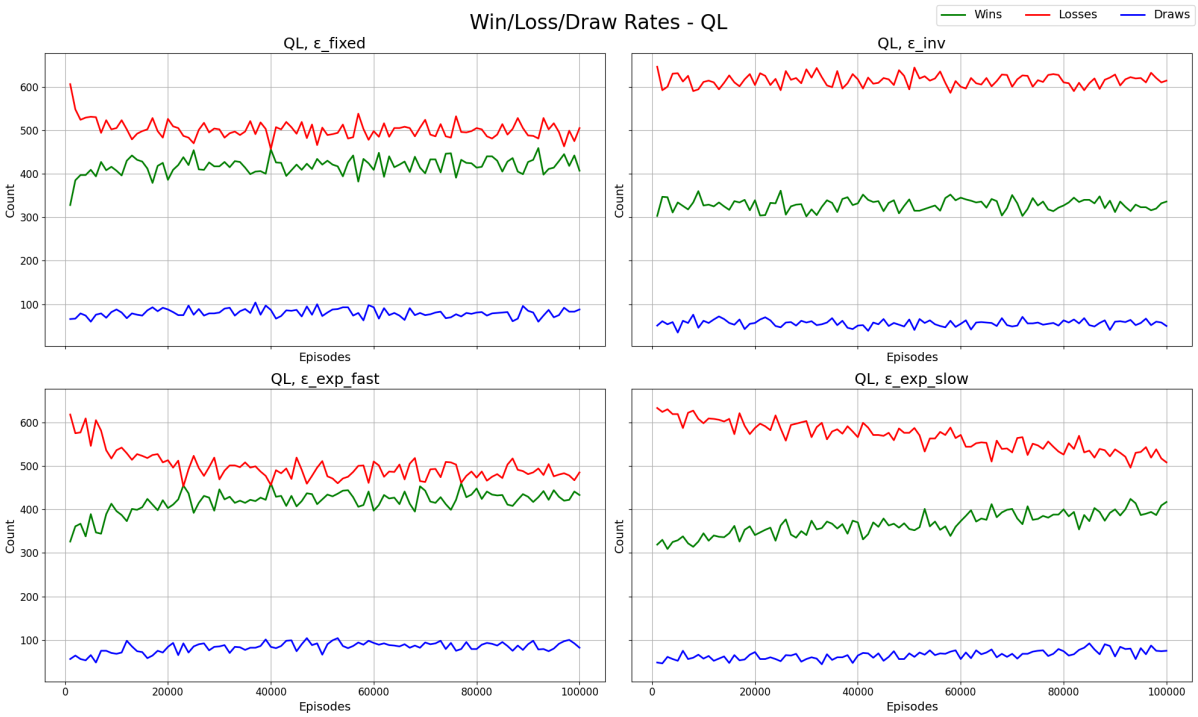

Fig.14 Win/Draw/Loss Rates for Q-Learning for all configurations

Fig.15 Win/Draw/Loss Rates for Double Q-Learning for all configurations

Dealer Advantage

To analyse which algorithm discovers the strongest policy, we can observe the dealer advantage across all configurations, as shown in Fig.16. From the plot, we concluded that the best-performing configuration was Monte Carlo with Exploring Starts with fast exponential decay. Additionally, the next best-performing configurations include versions of all four evaluated algorithms; MC, SARSA, Q-learning, and Double Q-learning, each using ε_fixed and ε_exp_fast. These results indicate that with appropriately tuned exploration, all four algorithms can learn similarly effective policies for this task.

Moreover, the fact that a relatively simple algorithm like MC-ES performed on par with more complex methods such as SARSA and DQL highlights the accessible nature of the Blackjack environment. Mainly because Blackjack has a well-defined and limited state-action space, which reduces the advantage that function approximation methods typically offer in larger or continuous domains. On the other hand, ε_inv and ε_exp_slow, especially Q-learning with ε_inv, performed the worst, yielding the highest dealer advantage of over 0.30. This implies that maintaining high exploration late into training may prevent the agent from consistently exploiting learned value estimates. Overall, rapid decay strategies enable faster convergence to stronger policies across configurations.

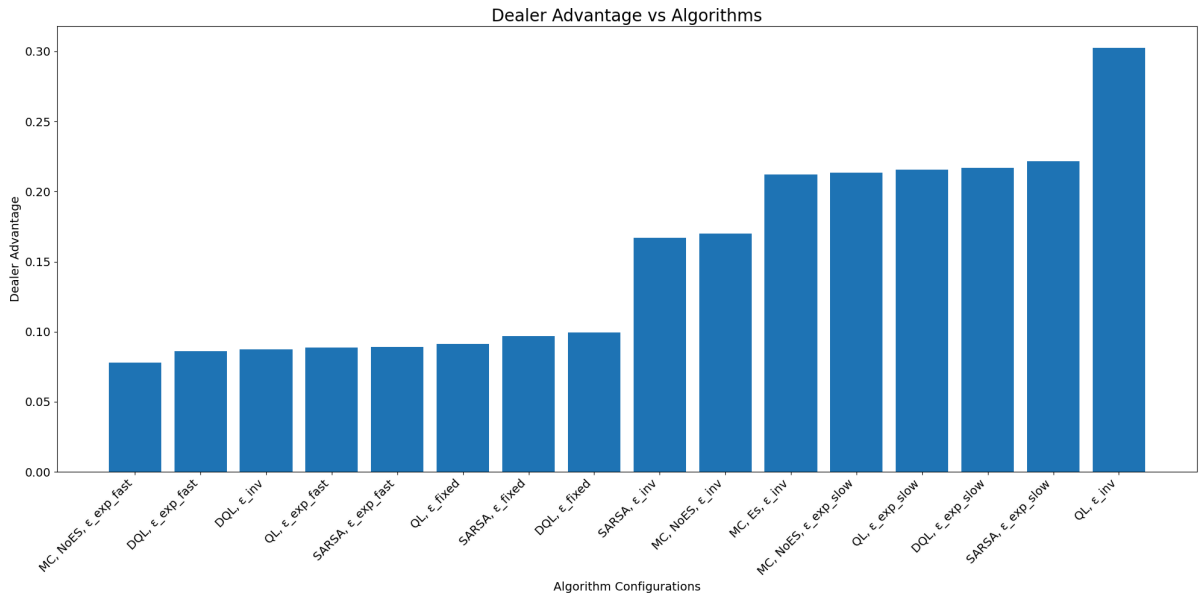

Fig.16 Dealer advantage plot for all algorithms and configurations respectively

Conclusion

This assignment implemented four fundamental reinforcement learning algorithms: Monte Carlo, SARSA, Q-Learning and Double Q-learning in a Blackjack environment. A complete simulation framework was developed from scratch including environment modelling, custom Q-table management and flexible policy control. All agents were trained using ε-greedy policies under various decay configurations where their performance was evaluated over 100,000 episodes using metrics such as state-action coverage, strategy stability and dealer advantage.

Across all configurations, the agents successfully explored the entire state-action space and converged toward expected Blackjack strategies where the agent stands on high totals and selectively hits with mid-range totals, especially when a usable ace is present. Strategy tables confirmed that all algorithms were able to learn policies that also align with conventional Blackjack decisions. Fast exponential decay (ε_exp_fast) often provided the best balance between early exploration and late-stage exploitation, resulting in stronger convergence and higher win rates across all methods.

Among the algorithms, Double Q-learning demonstrated the ability to generalise while reducing overestimation bias observed in Q-learning. However, simpler approaches like Monte Carlo with exploring starts performed comparably well showing the effectiveness of episodic learning in structured environments like this. Overall, the results emphasise the importance of analysing different strategies to approximate the most optimal plays in probabilistic, sequential decision-making problems.

Generative AI Usage

Introduction

ChatGPT's GPT-4o model was mainly used to assist with coding, problem-solving, and improving the overall quality of the project.

AI Usage

Generating a strategy table
ChatGPT 4o
Person: Matthew Pirotta

Generative AI was utilized to produce the necessary code for generating an informative and visually appealing plot. Although I had never created such a plot before, this has given me the understanding to create similar plots independently in the future.

> Prompt:
> Using the Q-values estimated for each state, where a state is defined as the tuple (player_total, dealer_card, usable_ace), and Q is initialized as self.Q = defaultdict(lambda: {'hit': 0.0, 'stand': 0.0}). Determine the optimal action (either 'hit' or 'stand') for every state. Based on this, construct a Blackjack strategy table with the dealer's visible card (2 through 10 and Ace) as columns and the player's total (from 20 down to 12) as rows. Each cell should contain 'H' for hit or 'S' for stand, indicating the best action in that state. Generate two strategy tables per algorithm configuration: one for when the player has a usable Ace (i.e., Ace counted as 11), and one for when they do not. Make the 'Hit' cells red, and the 'Stand' cells blue.

Debugging

1. ChatGPT 4o
   Person: Matthew Pirotta

   While running the data visualization script, FutureWarnings appeared about deprecated downcasting behavior in pandas' replace() method. The warnings recommend using .infer_objects(copy=False) or adjusting pandas options to maintain compatibility with future versions.

   Prompt:
   I am getting the following warning
   …

Would it be best to use .infer_objects(copy=False) or to adjust the global setting? Give me a detailed explanation of the pros and cons of both.

Results and suggestions for improvement

2. ChatGPT 4o
   Person: Daniela Curmi

   After finalising the analysis section of our report, it was crucial to evaluate the quality of our results and evaluations. Hence, AI was used as a proofreading tool, offering suggestions and indicating some limitations in our work.

   Prompt:
   Now that I've finalised my analysis on the 'X' algorithm, can you offer some suggestions and improvements to my current evaluation of results, and if my results can be improved?

Expected results per algorithm

3. Gemini 2.5
   Person: Daniela Curmi

   In this case, AI was used to get a bit of background information and a better idea of what results we should expect per algorithm. This is beneficial for the overall quality of our report and was a helpful insight to have before discussing the results obtained in all our plots.

   Prompt:
   Before documenting my analysis on the graph plots I obtained for MC, SARSA, Q-Learning, and Double Q-Learning, can you give me a general description of what results I should get when outputting these plots, respectively?

Grammar and Vocabulary check

4. Gemini 2.5
   Person: Daniela Curmi

   This was used mainly after completing small sections in our documentation just as a grammar/vocabulary tool to double-check our wording and grammar in general. It was useful since after reviewing your own work for an extensive period of time, one might overlook some minor grammatical errors which can be easily caught using AI.

Prompt:
Can you check the grammar and vocabulary of this paragraph? Can it be written more cohesively and coherently?

Double Q-Learning – Algorithm Overview and Implementation

5. ChatGPT4o
   Person: Matthew Privitera

   Generated the motivation behind the Double Q-learning method and its theoretical formulation by using the dual Q-table update mechanism. The output was further refined and adapted with appropriate referencing to Hasselt's work [9].

   Prompt:
   Can you explain the main motivation behind Double Q-learning and how it addresses it?

Double Q-learning – Data Analysis

6. Claude 3.7 Sonnet
   Person: Matthew Privitera

   Helped interpret the strategy table and state-action count graphs by identifying any patterns or inconsistencies across the ε-configurations. Once the main idea of these graphs was understood, I drafted the initial analysis paragraphs and expanded further.

   Prompt:
   Can you help me analyse the behaviour of the agent based off the state-action count bar chart and the strategy table? Are there any general patterns or inconsistencies that I should look out for?

Critical Assessment

In this section, we will discuss the strengths, weaknesses, and relevance of GPT-4o's responses, followed by some reflective points about using AI as a learning tool.

Strengths
One of AI's main strengths is its ability to generate code both quickly and efficiently e.g. when we needed a deeper understanding of what results we should obtain for each respective algorithm. Additionally, the clarity and creativity of GPT-4o helped simplify complex concepts with detailed explanations and offer innovative visualizations e.g. graphs.

Weaknesses
On the other hand, AI has some drawbacks that came into play when working through the assignment. Something we struggled with was when the AI omitted details when we were in the process of debugging, causing even more errors. Some unjustified assumptions were made due to contextual gaps.

Relevance
In some cases, the code generated didn't fully align with the specific use case, making it impractical for direct implementation. In some instances, the code included redundant or unnecessary cases that did not contribute to the desired functionality. As a result, it required selective adaptation and modification to meet the specific requirements of the task at hand.

Conclusion

Reflection
In conclusion, generative AI was a valuable asset throughout the project, acting as a form of brainstorming and polishing of code. However, its effectiveness relied heavily on our ability to critically evaluate, adapt, and integrate its outputs. Overall, the process of using AI enhanced our understanding of graph theory, network analysis, and predictive modeling.

References

[1] A. G. Barto and R. S. Sutton, Reinforcement Learning: An Introduction, 2nd ed., MIT Press, 2018.

[2] N. Modha, "Reinforcement Learning: Optimal Blackjack Strategies," Medium, Jun. 14, 2024. https://medium.com/mitb-for-all/reinforcement-learning-optimal-blackjack-strategies-982a5a176d2d (accessed May 20, 2025).

[3] A. Arutyunov, "Win at Blackjack with Reinforcement Learning," The Power of AI, Feb. 07, 2023. https://medium.com/the-power-of-ai/blackjack-with-reinforcement-learning-95f588dd670c (accessed May 20, 2025).

[4] Bechir Trabelsi, "On-policy vs Off-policy Monte Carlo Control Methods for Supply Chain Optimization: A Use Case of Inventory Replenishment," Medium, Oct. 22, 2023. https://bechirtr97.medium.com/on-policy-vs-off-policy-monte-carlo-control-methods-for-supply-chain-optimization-a-use-case-of-7b8d0d2e80e6 (accessed May 20, 2025).

[5] *A. D, "Reinforcement Learning model for BlackJack," Medium, Dec. 17, 2022. https://medium.com/@Andrew_D./reinforcement-learning-model-for-blackjack-a29817218d53* (accessed May 20, 2025).

[6] "An introduction to Q-Learning: reinforcement learning," freeCodeCamp.org, Sep. 03, 2018. https://www.freecodecamp.org/news/an-introduction-to-q-learning-reinforcement-learning-14ac0b4493cc/ (accessed May 20, 2025).

[7] CodeEmporium, "Q-learning - Explained!," YouTube, Nov. 07, 2023. https://www.youtube.com/watch?v=TiAXhVAZQl8 (accessed May 20, 2025).

[8] B. Lin, E. Li, and N. Luxsuwong, "Optimizing Blackjack Strategy with Reinforcement Learning." Available: https://web.stanford.edu/class/aa228/reports/2020/final17.pdf (accessed May 20, 2025).

[9] H. Van Hasselt, "Double Q-learning," 2010. Available: https://proceedings.neurips.cc/paper_files/paper/2010/file/091d584fced301b442654dd8c23b3fc9-Paper.pdf (accessed May 20, 2025).

[10] J. Geiser and T. Hasseler, "Beating Blackjack -A Reinforcement Learning Approach." Available: https://web.stanford.edu/class/aa228/reports/2020/final117.pdf (accessed May 20, 2025).

Plagiarism Form

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY

Declaration

Plagiarism is defined as "the unacknowledged use, as one's own work, of work of another person, whether or not such work has been published" (Regulations Governing Conduct at Examinations, 1997, Regulation 1 (viii), University of Malta).

~~I~~ / We*, the undersigned, declare that the [~~assignment~~ / Assigned Practical Task report / Final Year Project report] submitted is ~~my~~ / our* work, except where acknowledged and referenced.

~~I~~ / We* understand that the penalties for making a false declaration may include, but are not limited to, loss of marks; cancellation of examination results; enforced suspension of studies; or expulsion from the degree programme.

Work submitted without this signed declaration will not be corrected, and will be given zero marks.

* Delete as appropriate.

(N.B. If the assignment is meant to be submitted anonymously, please sign this form and submit it to the Departmental Officer separately from the assignment).

Daniela Curmi	
Student Name	Signature
Matthew Privitera	M. Privitera
Student Name	Signature
Matthew Pirotta	
Student Name	Signature
Student Name	Signature

ARI2204	Reinforcement Learning Group Project
Course Code	Title of work submitted

21st April, 2025

Date

Distribution of Work

Assignment Sections	Matthew Privitera	Daniela Curmi	Matthew Pirotta
Black Jack Environment Setup	✓	✓	✓
Reinforcement Learning Framework	✓	✓	✓
Monte Carlo On-Policy Control	✓		✓
SARSA On-Policy Control		✓	
Q-Learning Off-Policy Control		✓	
Double Q-Learning Off-Policy Control	✓		
Plotting Evaluation			✓
Report	✓	✓	✓
Signature	M. Privitera		